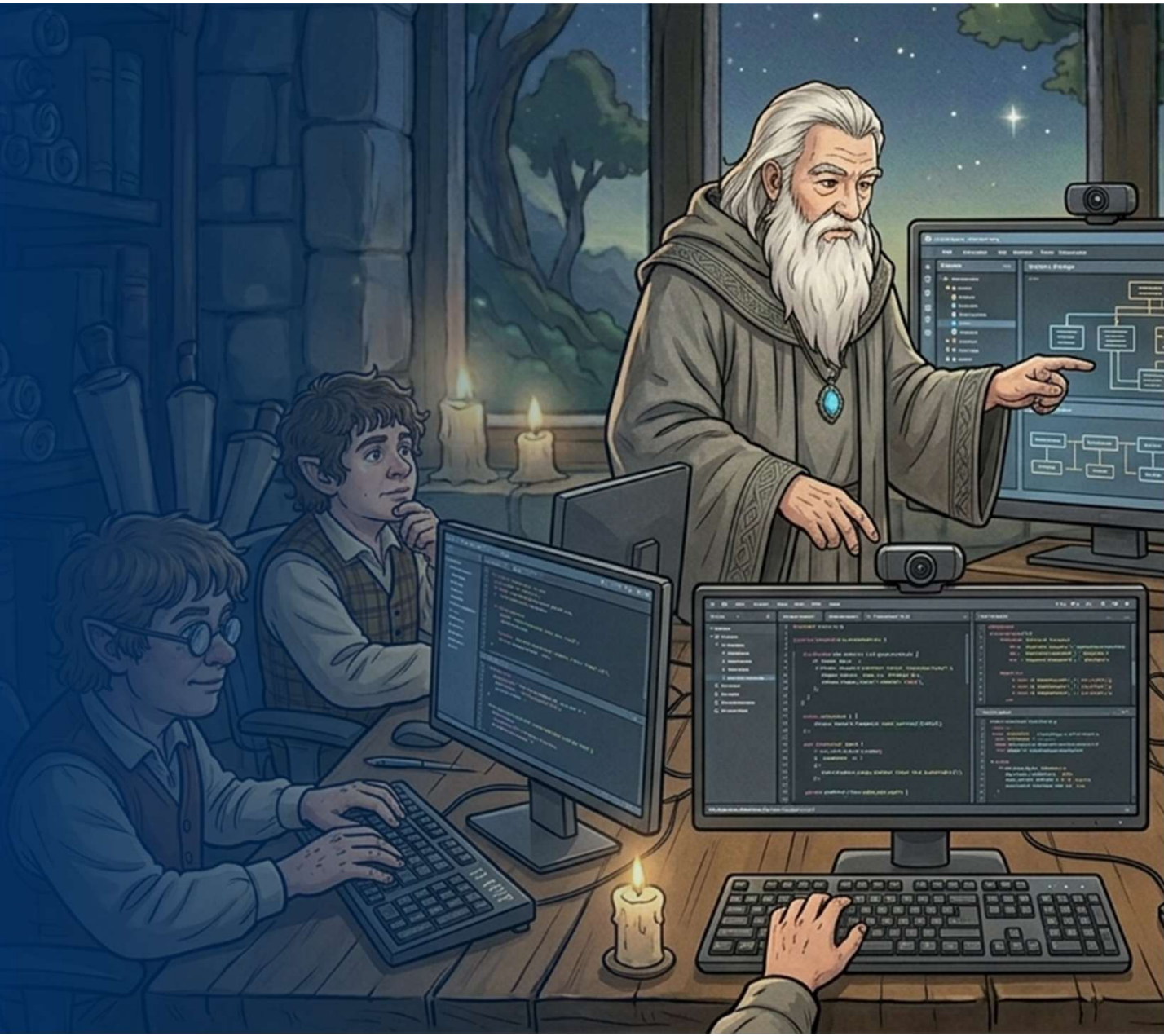
Application Design and Development *(ADaD)*

- The Fellowship of the Code

Phase 4: Application Synthesis

Connecting

Spring 2026, by Stefan Nafra



0.
Agenda

1. Feedback
2. Recapitulation
3. Theory
4. Practice
5. Recapitulation
6. Feedback

Feedback

1. Phase Review

1. Phase Review

„Despair is only for those who see the end beyond all doubt. We do not.“

- Gandalf

Recapitulation

2. Phase Recapitulation

2. Phase Recapitulation

1. Roadmap
2. Flashback
3. Feedback Management
4. Practice: TFC: Representation
5. Knowledge Recapitulation: Quiz
6. Community Challenge (Meta Game)

2.1. Roadmap

https://studiomuti.co.za/

2.2. Flashback

CONTENTS

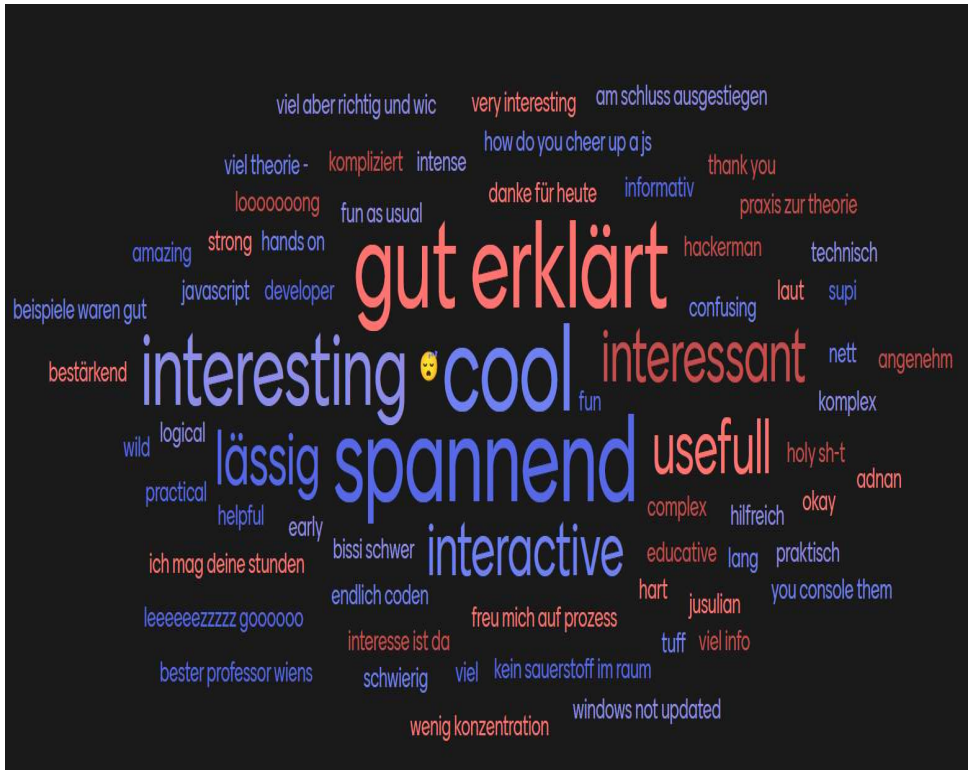
- Modelling
  - Development concepts
    - JavaScript
    - Tooling & structure

TAKEAWAYS

- Functionality of various programming concepts
  - Variables, Operators, Conditional statements, Loops and Functions
- Introduction and first steps in programming on the example of JavaScript

2.3. Feedback Management

- **Positive critics:** Thanks <3
- **Mixed critics:**
  - *“Viel Theorie” / “Viel Info”*
  - *“Kompliziert” / “Verwirrend” / “Intensiv” / “Schwierig”*
  - *“Wenn man HTML noch nicht kann, ist JS schwierig”*
  - *“Weniger Nitpicky sein” / “Java”*
  - *“Zeitmanagement” / “Teilweise sehr eng getaktet”*
  - *“Gruppenbewertung & Gamifizierung ist nicht gut für Benotungen“ / „Zu viel Gruppen, zu viel Abgaben“*
  - *“Wenig Konzentration“ / “Kein Sauerstoff im Raum“ / “Mehr in die Mitte des Raumes kommen“ / “Lärm im Raum“*
  - *“Mehr IT Security“*
  - *“Nicht immer dieselben Jokes und Geschichten erzählen“*



2.4. Practice: TFC: Logic & State

- Lessons learned:
  - *Read instructions: Do not waste valuable points by missing required details*
  - Separation of concerns: Do not use HTML instead of JS or JS for CSS
  - Order for conditions: Think of your order *(e.g., validations on top)*
  - Validations: HTML can easily be overridden (e.g., the disabled state)
  - Optimization: Use variables if you need a value multiple times
  - Style: Consistency and coding styles are important
  - UX: It requires a lot of thought and work *(e.g., predicting user behavior and enabling convenience)*

2.5. Knowledge Recapitulation: Code Review

„You shall not pass.“

- Gandalf

Recommended watching:

https://www.youtube.com/watch?v=3xYXUeSmb-Y

2.5.1. Justification

THIS IS:

- A validation of understanding
- A responsibility check
- A learning moment

THIS IS NOT:

- A coding exam
- A trick test
- A punishment for using AI

Our core principle: *You may use AI, peers, or tools - but you must understand what you submit*

If you can explain it, you own it

2.5.2. Review Flow

OWNERSHIP QUESTION

- Each team gets one question, tailored to their own submission
- They must:
  - Explain behavior, logic, or design intent
  - Reference their own code/artifact
  - Speak in plain language *(no buzzwords)*

OUTCOME

- If the team:
  - Explains clearly, can point to where this is expressed in the artifact, and shows understanding of intent, logic, or constraints
    - Nothing happens and points remain
  - If the answer is vague, incorrect, AI-generated without understanding
    - We ask the audience for an **assisted recovery**

2.5.3. Team FiresOfMordor

- Zeile #238 - #243 versus #250 – 254:
  - *„Was passiert hier und was ist der Unterschied in der Funktionsweise?“*
  - Im UI: *„Was muss der Nutzer bei den „klickbaren“ Elementen „einfach“ wissen?“*

- Joker: *„Welche der beiden Funktionen ist leichter zu erweitern und weniger fehleranfällig?“*
- *HALF*

2.5.4. Team Goldberries

- Zeile #79 – 85:
  - *„Was passiert hier?“*
  - *„Wie könnte man hier weitere Felder dieser Funktionalität hinzufügen und was würde es aber in dieser Hinsicht robuster machen?“*

- Joker: *„Was muss der Nutzer bei der Neuanlage eines Votings also „einfach“ wissen und wie könnte man es deutlicher machen?“*
- *VOLL*

2.5.5. Team GolumGPT

- Zeile #204 – 257:
  - *„Was passiert hier?“*
  - *„Was macht ‚return‘ hier, welches wichtige Programmierprinzip wird hier verletzt und wie könnte man das umgehen?”*

- Joker: *„Auf dieser Seite tritt keine Funktionalität ein, stattdessen wird eine Fehlermeldung angezeig: Welche Ursachen hat das und was muss angepasst werden?“*
- *HALF*

2.5.6. Team IsengardInnovations

- Zeile #5 - #12 & #26 – 33:
  - *„Was passiert hier?“*
  - *„Was ist der Unterschied in der Funktionsweise, wie diese Funktionalität aufgerufen bzw. gebunden wird?”*

- Joker: *„Wen man die Funktionalität um einen weiteren Button erweitern will, was braucht dieser Button dann zwingend?“*
- *NO*

2.5.7. Team OneAppToRuleThemAll

- Zeile #4 – 14:
  - *„Was passiert hier?“*
  - *„Was passiert hier für den „normalo“ Nutzer im Fehlerfall und wie erkennt er das?”*

- Joker: *„Welche Möglichkeiten gäbe es, um dem Nutzer den Fehlerfall auch ersichtlich zu machen (abgesehen von der Konsole)?“*
- *HALF*

2.5.8. Team OneDoesNotSimplyDevelopAnApp

- N/A

2.5.9. Team TheBugHuntersOfMiddleEarth

- Zeile #79 – 88:
  - *„Was passiert hier?“*
  - *„Was ist der Unterschied bei den zwei Funktionalitäten und welche ist ‚sauberer‘ (dry)?“*

- Joker: *„Was fehlt nach einem deaktivierten Alarm in Bezug auf die Funktion zum Ausschalten des Alarms?“*
- *HALF*

2.5.10. Team TheFellowshipOfTheFive

- Zeile #117 – 137:
  - *„Was passiert hier?“*
  - *„Was macht ihr mit diesen DOM-Helfern? Wann sind sie nützlich und wann eher nicht?“*

- Joker: *„Bezogen auf das UI (UX), was fehlt bei den Actions (Buttons) ganz dringend?“*
- *HALF*

2.5.11. Team WeShallPass

- Zeile #14 – 32:
  - *„Was passiert hier?“*
  - *„Was macht ‚[…].style[…]‘ hier? Was sind diese ‚#6b7356‘-Werte? Was für eine Trennung halten wir hier nicht ein und wie könnte man das besser lösen?“*
- Joker: *„Was für eine Auswirkung hat ‚isAuthorized‘?“*
- *HALF*

2.5.12. Team YouShallNotDebug

- Zeile #74 – 77:
  - *„Was passiert hier?“*
  - *„Eure Eventkette ist ‚Ort ausgewählt‘ -> ‚updateState‘ -> ‚updateUI‘; Was ist das Problem in Bezug auf eure Validierung?“*
- Joker: *„Was fehlt der UI nach dem Auswählen einer Option zwingend (um es sichtbarer zu machen)?“*
- *HALF*

2.5.13. Team Shadowcoders

- *HALF*

Break Time

- *“Gentleman, we do not stop before nightfall”*
- *“What about breakfast?”*
  - *“We already had it.”*
- *“We had one, yes. What about second breakfast?”*

Recommended watching:

https://www.youtube.com/watch?v=gA8LV37QwxA

Theory: Integration Concepts

3. Libraries & Frameworks
4. Third-Party Tools & Automation
5. Application Programming Interface (API)

3.

Libraries & Frameworks

1. Introduction
2. Libraries
3. Frameworks
4. Libraries versus Frameworks
5. Advantages & Disadvantages
6. Examples

3.1. Introduction

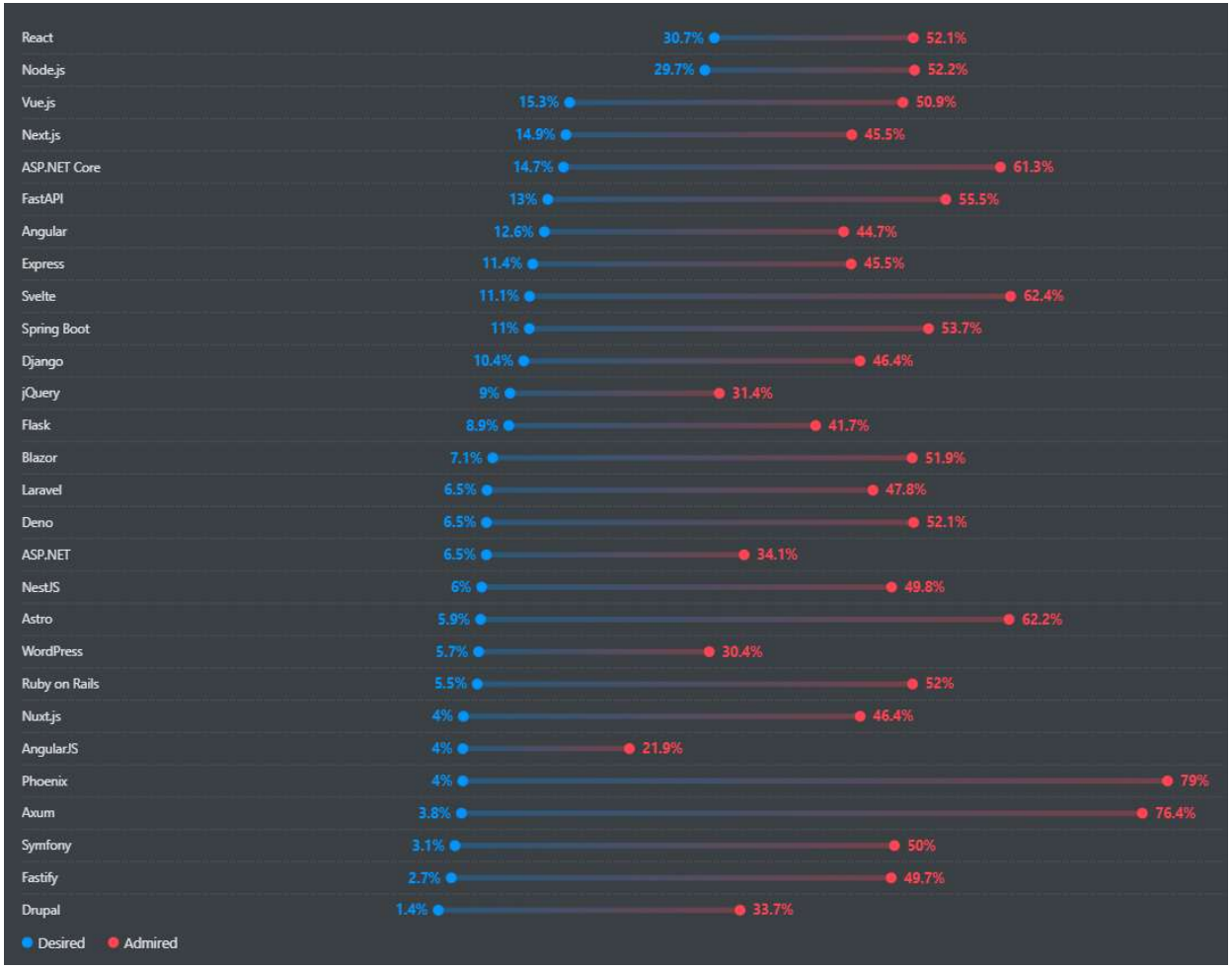
- Frameworks and libraries provide abstraction and modification *(simplification, harmonization, …)* to the complexity of certain functionality
  - Easier and more convenient use of native functionality
  - Provide fallbacks and situational fixes and helpers when addressing specific problems
  - Introduce standards and defaults
    - Guide and unify different code styles
    - Enable well-defined design patterns

3.2. Libraries

- Provide reusable code for specific areas and simplify certain tasks
  - **Simplest form:** Provides features *(= functions)*
- Reduce the amount of code and *(can)* speed up the delivery rate
- Usually come without logic
  - Developer is still in control of the flow and must decide when to do what

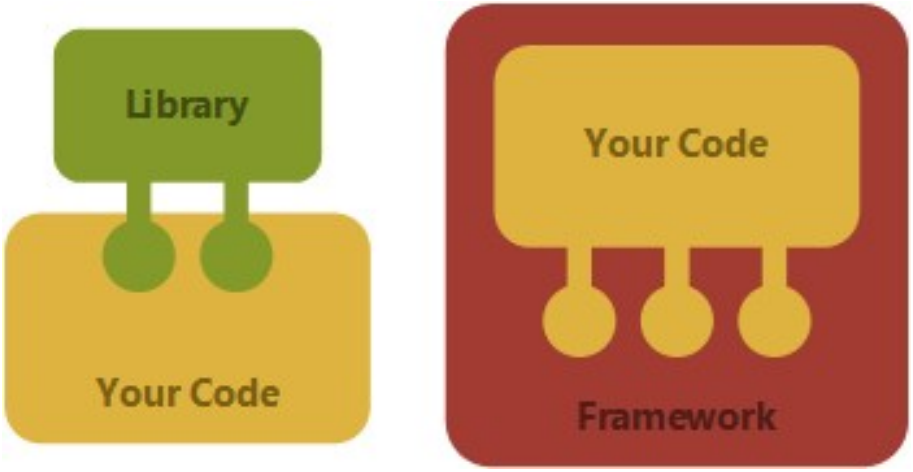
3.3. Frameworks

- Define an architecture that embodies an abstract design
- Provide a skeleton and certain rules and guidelines to control and structure the application
- Allow to concentrate on the specifics of the application itself rather than focusing on overhead
  - Handle the design decisions that are common and well known
- Usually come with logic
  - Define the flow and maintain quite some control over the implementation
  - Developer specifies own code and logic within the frameworks pre-defined places



https://survey.stackoverflow.co/2025/technology#2-web-frameworks-and-technologies

3.4. Libraries versus Frameworks



Recommended reading: *https://medium.com/iquii/applications-vs-frameworks-vs-libraries-c1f1a6122711/ & https://medium.com/@sofienebenkhemis/what-is-the-difference-between-a-framework-and-library-2b712a1a1c41/*

3.5. Advantages & Disadvantages

ADVANTAGES

- Use of patterns and structured way of coding
- Unified programming styles and flows
- Hidden complexity
- Fixed glitches and shortcomings of native implementations *(e.g., cross-browser combability)*

DISADVANTAGES

- Additional payload *(extra code impacting the loading time and performance)*
- Increased complexity and need for specific knowledge
- Dependencies *(require maintenance)*

3.6. Examples

- jQuery *(JavaScript library)*
- Bootstrap *(CSS library)*
  - *Modern? No!*
    - *Take Tailwind instead of Bootstrap and Vanilla JS instead of jQuery*

3.6.1. jQuery

- *jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers. With a combination of versatility and extensibility, jQuery has changed the way that millions of people write JavaScript.* [1]

[1] https://jquery.com/

script.js

```
1   $(function () {
2       $('.row').fadeOut();
3
4       $('.row').each(function () {
5           $(this).after('<h2>Headline</h2>');
6       });
7
8       $('p').click(function () {
9           $(this).attr('style', 'color: red');
10      });
11  });
```

3.6.2. Bootstrap

- *Build fast, responsive sites with Bootstrap* [1]
  - *Quickly design and customize responsive mobile-first sites with Bootstrap, the world's most popular front-end open-source toolkit* [1]
- Features: [1]
  - Responsive grid system
  - Prebuilt components
  - JavaScript plugins

[1] https://getbootstrap.com/

```
10  <body>
11      <div class="row text-center">
12          <div class="col-6">
13              <p>Left Side</p>
14          </div>
15          <div class="col-6">
16              <p>Right Side</p>
17          </div>
18      </div>
```

4.

Third-Party Tools & Automation

1. Introduction
2. Automation Is Still Development
3. Why We Use Automation

4.1. Introduction

THIRD-PARTY TOOLS

- External services you connect to your system
- They expose capabilities you don’t want to build yourself
- Examples:Email services, file storage, calendars, databases, AI services

AUTOMATION

- A defined flow of actions
- Trigger ⟶ Logic ⟶ Outcome
- Runs without human intervention

4.2. Automation Is Still Development

CLASSIC

- Inputs & outputs
- Conditions (if/else)
- Errors & edge cases
- State & side effects

CHANGING

- Less syntax, more configuration
- Visual flows instead of text files
- Platform constraints instead of compiler rules

Automation is just code you didn't write, executing logic you defined, triggered by events

4.3. Why We Use Automation

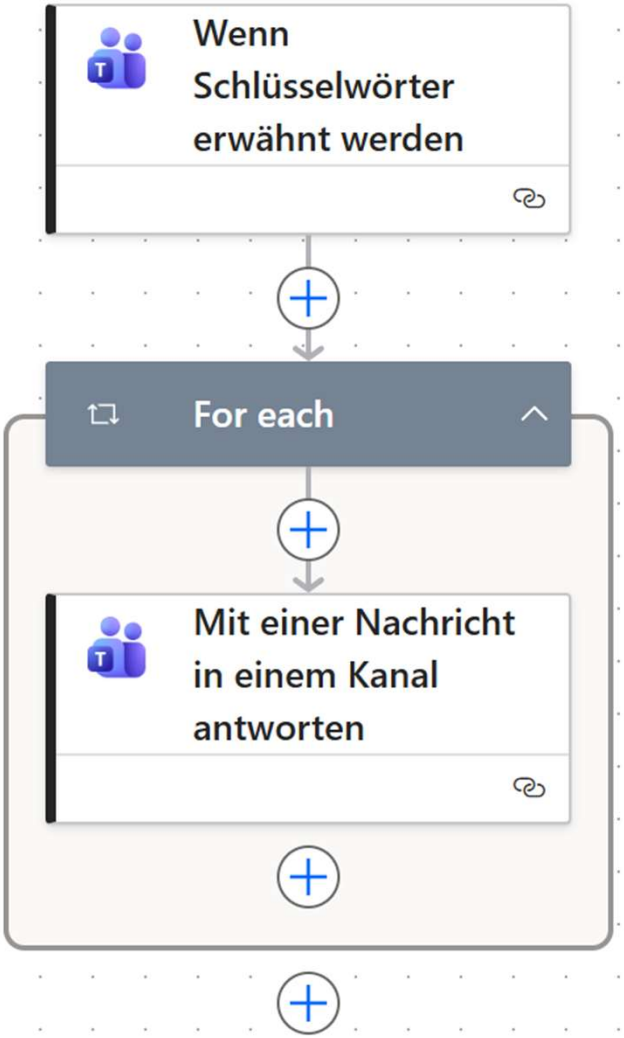
GOOD REASONS

- Remove repetitive work
- Reduce human error
- Integrate systems quickly
- Prototype workflows before building custom code

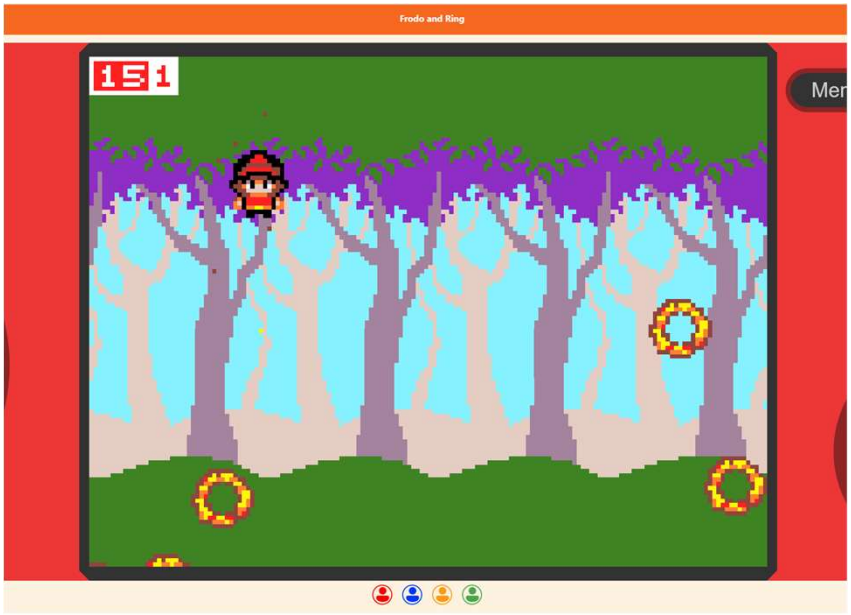
BAD REASONS

- “It works, so I didn’t look deeper”
- Blind copy-paste from AI
- Not knowing failure cases

If you cannot explain the flow step-by-step, you do not understand it - even if it works



Recommended reading: *https://make.powerautomate.com/*



Recommended reading: *https://arcade.makecode.com/S40163-03284-17870-68641 & https://arcade.makecode.com/S46013-36108-23991-23193*

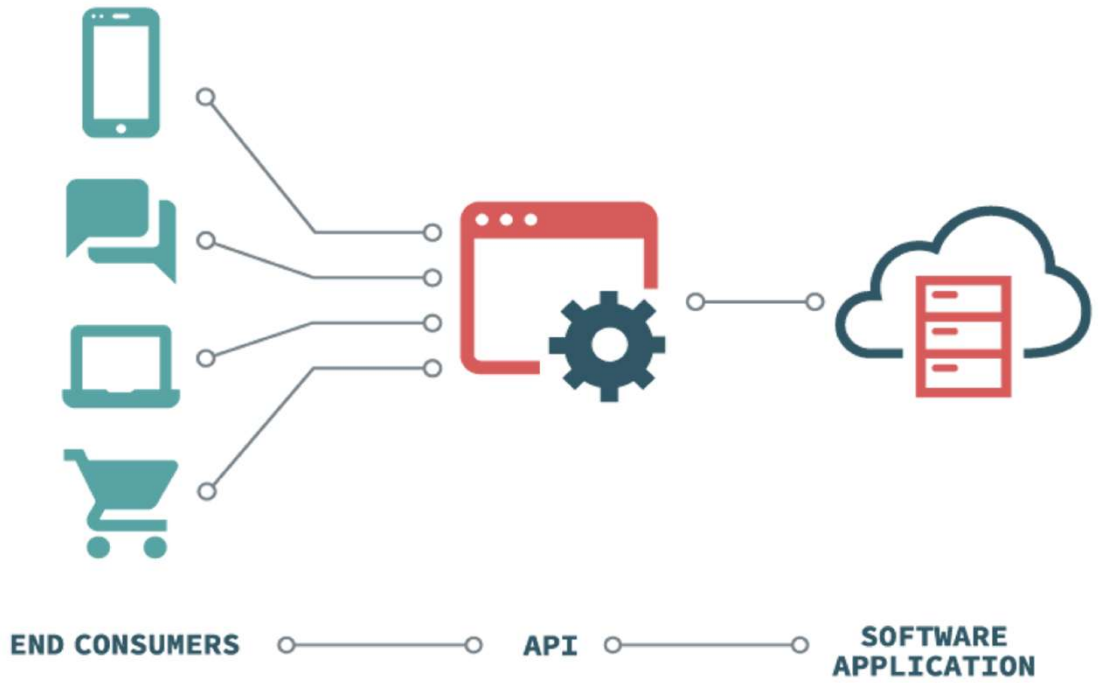
5.

Application Programming Interface (API)

1. Introduction

5.1. Introduction

- Provides access to existing sources of data or services
  - Instructions and standards *(request details, data formats, …)*
  - Usually defines the HTTP request and response messages using a JSON or XML structure
    - Using endpoints *(location of resources)*
- Principle of information hiding
  - Hiding implementation details and complexity of underlying systems *(providing only the needed information)*
- **Documentation:** Multiple standards exists *(e.g., OpenAPI for REST APIs)*



Source: *https://www.aloi.io/docs/installation/*

Break Time

- *"Gentleman, we do not stop before nightfall"*
- *"What about breakfast?"*
  - *"We already had it."*
- *"We had one, yes. What about second breakfast?"*

Recommended watching:

https://www.youtube.com/watch?v=gA8LV37QwxA

Practice

5. Application Programming Interface (API)

5.

Application Programming Interface (API)

1. Setting
2. Guided API Demo
3. Rules & Rewards

* = *Focus*

5.1. Setting

- In-Class API Hackathon:
  - *“The World Beyond the Shire“*
- Goal:
  - Experience API integration *(not mastery)*
    - See how external data can influence an application
    - Understand the shape of an API *(request - response - use)*
    - Practice reading API documentation
    - Experience integration friction *(auth, data formats, limits)*
    - Learn what not to over-engineer

5.2. Guided API Demo

- I'll guide you through a basic example
  - https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/api.html

Recommended reading: https://estebanthewhite.github.io/The-FellowShip-of-the-Code-2026/materials/JavaScript/api.html

5.3. Rules & Rewards

RULES

- One API per team and one small output
  - No new features beyond “data influences something”
  - AI assistance is NOT allowed
  - If you understand what you tried and why it would work, you did it right

REWARDS

- Everyone leaves with:
  - One working example or
  - One clear understanding of why it didn’t
- The fastest team that can correctly solve the problem and explain their solution will not have to complete the next side quest *(homework)*

Break Time

- *“Gentleman, we do not stop before nightfall”*
- *“What about breakfast?”*
  - *“We already had it.”*
- *“We had one, yes. What about second breakfast?”*

Recommended watching:

https://www.youtube.com/watch?v=gA8LV37QwxA

Recapitulation

6. Lecture Recapitulation
7. Self-Study Assignments

6.

Lecture Recapitulation

1. Flashback
2. Quiz

6.1. Flashback

CONTENTS

- Application synthesis
  - Integration concepts
    - Libraries & Frameworks
    - Third-Party Tools & Automation
    - Application Programming Interface (API)

TAKEAWAYS

- Functionality of various integration concepts

6.2. Quiz

„You shall not pass.“

- Gandalf

Recommended watching:

https://www.youtube.com/watch?v=3xYXUeSmb-Y

6.2.1. Libraries & Frameworks: What are the advantages and disadvantages?

ADVANTAGES

- Use of patterns and structured way of coding
- Unified programming styles and flows
- Hidden complexity
- Fixed glitches and shortcomings of native implementations *(e.g., cross-browser combability)*

DISADVANTAGES

- Additional payload *(extra code impacting the loading time and performance)*
- Increased complexity and need for specific knowledge
- Dependencies *(require maintenance)*

6.2.2. Why do we use automation?

GOOD REASONS

- Remove repetitive work
- Reduce human error
- Integrate systems quickly
- Prototype workflows before building custom code

BAD REASONS

- “It works, so I didn’t look deeper”
- Blind copy-paste from AI
- Not knowing failure cases

6.2.3. What is an Application Programming Interface (API)?

- Provides access to existing sources of data or services
  - Instructions and standards *(request details, data formats, …)*
  - Usually defines the HTTP request and response messages using a JSON or XML structure
    - Using endpoints *(location of resources)*
- Principle of information hiding
  - Hiding implementation details and complexity of underlying systems *(providing only the needed information)*
- **Documentation:** Multiple standards exists *(e.g., OpenAPI for REST APIs)*

7. Self-Study Assignments

#	TITLE	TYPE	DUE DATE	TASKS	TOOLING
15	Extending the Fellowship	PRAC	23.06.2026	• Connect the system	• https://make.powerautomate.com/ • https://arcade.makecode.com/
16	TFC: Integration & Extension	PRAC	23.06.2026	• Integrate and extend the system	• GitHub & GitHub Codespaces
17	Class Report	PRAC	23.06.2026	• Provide structured course feedback	• Any text processing tool
18	TRDoW: Chapter V - The Tale Continues	PRAC	29.05.2026	• Document & reflect • Share & care • Prepare	• GitHub & GitHub Codespaces

Feedback

8. Lecture Review

8. Lecture Review

„Despair is only for those who see the end beyond all doubt. We do not.“

- Gandalf